Anthony Nuth
Kenneth Vuong
CECS 478
Final Project Report

NetSentry Final Report

1. Problem Statement

Modern systems are constantly exposed to network-based threats such as reconnaissance scans, repeated unauthorized access attempts, and malicious web requests. Even in educational or small-scale environments, security tools are often demonstrated in ways that are difficult to rebuild, verify, or compare across systems. This makes it harder to prove that a security claim is supported by repeatable evidence rather than a one-time demo.

NetSentry addresses this problem by providing a reproducible, Docker-based intrusion detection evaluation pipeline using Suricata, controlled PCAP-driven demo inputs, structured logs, and exported summary metrics. Rather than attempting to build a new intrusion detection engine from scratch, the project focuses on creating a repeatable workflow that can be rebuilt, rerun, and evaluated consistently. This is relevant because reproducibility, observability, and evidence-backed claims are essential in security engineering, especially when demonstrating detection behavior in a controlled academic environment.

2. System Design

NetSentry uses a containerized architecture built around Docker Compose. The system includes a mock web service, a Suricata IDS container, a replay/demo pipeline, and an evaluation/export stage.

The mock web service is implemented as a lightweight Python HTTP server and serves as a simple network-facing application for the environment. Suricata is configured through a project-specific YAML configuration file and local rule set. These files are mounted into the container so that the IDS configuration and detection logic are version-controlled and reproducible.

The demo pipeline is driven through Makefile targets and shell scripts. The main workflow uses make clean && make up && make demo. The make up target starts the Docker environment, including the mock web service and the Suricata container. The make demo target invokes a demo suite that processes both benign and malicious PCAP-driven scenarios, generates

structured eve.json-style output, exports JSON summaries, and produces comparison artifacts for evaluation.

The evaluation stage is implemented in Python. The evaluator parses the generated log file, counts processed events and alert events, and exports the results as JSON. Additional comparison artifacts are generated as CSV and markdown tables to support the report and demo. Release artifacts are stored under artifacts/release/ so that the final result can be reproduced and inspected later.

A simplified architecture is:

PCAP-driven demo input -> replay/demo script -> Suricata-configured environment -> log generation -> evaluation script -> metrics and release artifacts

3. Threat Model

NetSentry assumes an external attacker without privileged access is interacting with a monitored service. The project focuses on detection-oriented security rather than prevention. The main assets are the integrity of the generated logs, the correctness of the exported metrics, the reproducibility of the pipeline, and the visibility provided into suspicious traffic behavior.

The attacker model includes suspicious or malicious traffic represented by controlled demo inputs. In the released implementation, the primary comparison is between benign traffic and malicious traffic. The attack surface includes the monitored application service, the replay/demo pipeline, the Suricata configuration and rules, and the artifact-generation path.

Trust boundaries exist between the host system, the Docker environment, the mock service, and the generated artifacts. NetSentry assumes that the Docker environment is trusted to run the configured services, that the local project files have not been tampered with, and that the approved PCAP inputs under the project directories are the only accepted replay inputs. The system also assumes a controlled academic environment rather than an enterprise production deployment.

NetSentry is detection-focused and does not claim to prevent attacks in real time. It also does not claim complete coverage against stealthy, novel, or large-scale real-world attacks. Its security posture is limited to reproducible detection/evaluation behavior for selected controlled scenarios.

4. Methods

The system was implemented using Docker Compose, a Python-based mock web service, Suricata configuration files, local Suricata rules, shell-based replay/demo scripts, and a Python

evaluation script. Development focused on making the system reproducible and easy to validate from a clean environment.

The project workflow begins with make clean, which removes previous container and output state. make up builds and starts the Docker stack. make demo executes the demo suite, which runs both a benign scenario and a malicious scenario. For each case, the replay/demo script validates the PCAP path, ensures the file is located in an approved directory, ensures the extension is .pcap, and then processes the selected input. This input validation serves as the project's primary hardening measure.

The replay/demo pipeline generates structured log output in eve.json format. The evaluation script reads that log file and exports a JSON summary containing the selected PCAP, the number of events processed, the number of detected alerts, and the status of the run. The demo suite then saves separate summary files for the benign and malicious cases and generates comparison artifacts in both CSV and markdown table form.

Testing was implemented using a Python test file covering a happy-path malicious case, a missing-file negative case, a wrong-extension negative case, and a benign case that should produce zero alerts. A GitHub Actions workflow was added to run the test suite and provide coverage output in CI. This ensures that the project is not only runnable but also automatically checked.

5. Results

The final release demonstrates a reproducible end-to-end pipeline that distinguishes between benign and malicious demo inputs. The main result is produced through make clean && make up && make demo, which generates both per-scenario summaries and a comparison artifact.

The benign scenario produced the following result:

- events processed: 1
- alerts detected: 0

The malicious scenario produced the following result:

- events processed: 4
- alerts detected: 3

These results are summarized in the following table:

Traffic Type	Events Processed	Alerts Detected

Benign	1	0
Malicious	4	3

These outputs show that the current NetSentry release can distinguish between the two controlled traffic classes at a basic level. The benign scenario completes successfully without generating alerts, while the malicious scenario generates multiple alerts and a higher event count. This supports the claim that the pipeline is functioning as a repeatable IDS-style evaluation workflow rather than a static configuration-only demo.

The main evidence artifacts referenced in this report include:

- artifacts/release/metrics/summary_benign.json
- artifacts/release/metrics/summary_malicious.json
- artifacts/release/charts/alert_counts.csv
- artifacts/release/charts/alert_counts.md
- artifacts/release/logs/eve.json

Together, these artifacts support the final demo, the report claims, and the reproducibility pack.

6. Limitations

NetSentry has several important limitations. First, the released implementation uses a controlled Docker-based IDS evaluation pipeline rather than a full production-grade live intrusion detection deployment. The final workflow relies on controlled PCAP-driven demo inputs and exported artifacts rather than full live packet replay into a hardened enterprise monitoring environment.

Second, the attack coverage is limited. The final release demonstrates a benign-versus-malicious comparison rather than a broad multi-category evaluation across many real-world attack types. The project is therefore best understood as a reproducible academic prototype and evaluation harness, not a comprehensive IDS benchmark.

Third, NetSentry does not prevent attacks. It is detection-focused and demonstrates how logs, metrics, and artifacts can be generated reproducibly from selected scenarios. It does not claim to stop compromise, block malicious traffic, or provide full real-time defense.

Finally, the project depends on the assumptions of the Docker lab environment and approved project inputs. It is not intended to model the full complexity of production traffic, stealthy adversaries, or adversarial evasion beyond the scope of the chosen demo scenarios.

7. Future Work

Several next steps would strengthen the project. One improvement would be to expand the PCAP corpus to include more traffic classes and a larger regression suite. Another would be to deepen the baseline-versus-tuned rule comparison so that false positives and detection coverage can be measured more systematically.

A second major improvement would be to explore a more realistic live replay path or richer traffic replay integration, while preserving reproducibility. This would bring the system closer to a live IDS sensor model while still allowing evidence-backed comparison.

Additional future work includes richer visualizations, more detailed CSV exports, more application-layer scenarios, improved alert classification, and stronger automation around frozen release artifacts. These changes would make NetSentry more useful as both a teaching tool and a more capable reproducible IDS evaluation framework.

Conclusion

NetSentry demonstrates that a reproducible Docker-based IDS evaluation pipeline can be built with Suricata, controlled PCAP-driven inputs, structured logs, and exported summary metrics. The final release provides a clean end-to-end workflow, a documented reproducibility path, evidence artifacts, and a controlled comparison between benign and malicious scenarios. While the project does not claim production-grade coverage or real-time attack prevention, it succeeds as a repeatable academic prototype that supports evidence-backed security evaluation and clear reproducibility guarantees.